Guion de los expositores — versión de 25 minutos

A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting (Euro-Par 2024)

Reparto y tiempos

Los tiempos están solo en este guion; las diapositivas no los muestran.

Expositor	Diapositivas	Tiempo
Apertura (David)	1–3	1,5 min
David — el problema	4–12	≈ 7,5 min
Hugo — la solución	13–21	≈ 8 min
Diego — corrección, evaluación y conclusiones	22–32	≈ 8 min
Preguntas	33 (34 es de reserva)	resto

Archivo: `exposicion_25min.pdf` (34 diapositivas). Si vais justos: acortad la 10 (contribuciones, leerla de corrido), la 25 (solo la tabla) y la 30.

Antes de empezar

- PDF a pantalla completa. Demos opcionales: `bash codigo/ejecutar_pruebas.sh` (desde `codigo/`) y `node simulador/buscar.js 20000 ingenua,sinReserva` (desde `simulador/`); si falla, enseñad la captura.
- Analogía: **libro** = objeto, **hoja** = contador, **pase** = referencia (peso 1), **cartel** = *atomic shared pointer*, **talonario** = peso, **tirar el libro** = liberar.

Apertura — David (1,5 min)

1. Portada — «Exponemos un paper de Euro-Par 2024 sobre conteo de referencias concurrente.» **2. Plan** — Yo el problema (con una demo de la carrera), Hugo la solución (con un simulador) y Diego la corrección, los experimentos y nuestras pruebas en local. **3. Analogía** — Recorrer la tabla despacio. Frase: «el cartel lleva un talonario de tickets ya apuntados en la hoja; leer es arrancar un ticket».

Parte 1 — David (≈ 7,5 min)

4. El paper en una frase (40 s) — Leer un puntero compartido con conteo **mientras otro hilo lo cambia**, sin candados, sin esperas y sin memoria extra. Mejora el *split reference count* (Folly, GCC), solo *lock-free*; el paper lo hace *wait-free*. Sello *Artifact Evaluated*: hay código público y lo usamos.

5. Conteo de referencias y atomic shared pointer (50 s) — Copiar suma 1, destruir resta 1, a 0 se libera en ese momento; con rc = 1 se modifica en el sitio. Shared pointer = inmutable;

atomic shared pointer = celda mutable con `load`/`store` (`std::atomic<std::shared_ptr<T>>`). El problema nace al leer esa celda.

6. La carrera read-reclaim (60 s) — Seguir las flechas: Ana lee A ⟶ la pausan ⟶ Beto pone B, resta uno a A (rc = 0) y **libera A** ⟶ Ana suma 1 a un libro que ya no existe. Dos pasos no atómicos; la ventana dura nanosegundos: pasa las pruebas y falla en producción.

7. ¿Por qué no en un solo paso? (50 s) — Puntero y contador viven en dos sitios. Existen `fetch_and_add`, `swap`, CAS y CAS de doble anchura (palabras contiguas); no existen DCAS ni `fetch_and_add` de 128 bits. Una atómica abarca una palabra.

8. Garantías de progreso (50 s) — Bloqueante: alguien puede esperar siempre. *Lock-free*: siempre avanza *algún* hilo (bucle CAS; yo podría fallar siempre). *Wait-free*: **cada** hilo acaba en un número acotado de pasos propios. Importa porque la gestión de memoria está en todas partes.

9. Lo que había (50 s) — Recorrer por columnas: *wait-free*, sin memoria extra, hardware actual, precio. Sundell es *wait-free* pero cada CAS cuesta $\Theta(P)$; Detlefs necesita DCAS; los de reclamación diferida gastan memoria; el *split refcount* es *lock-free*. Ninguno cumple las cuatro.

10. Contribuciones (35 s) — *Wait-free* `load`/`store`/`swap` sin memoria extra; `store` O(1) y `load` casi siempre una instrucción (O(P) peor caso); CAS y *aliasing* solo *lock-free*; reexplica el *split refcount* como conteo ponderado; experimentos.

11. Demo 1: AddressSanitizer (55 s) — Arriba, ingenua con 8 hilos: `heap-use-after-free`, el `WRITE` en `Ingenua::load()` (el `fetch_add`) y `freed by thread T2 … Ingenua::store()`: justo la diap. 6. Abajo, código de los autores en las mismas condiciones: 1 600 000 operaciones, 56 ms, sin errores.

12. La idea (45 s) — Cartel = (libro A, talonario 104); hoja = 106 = 104 + 1 + 1. Los tickets ya están pagados; leer = un `fetch_and_add` que arranca un ticket y dice qué libro hay. Leer y apuntarse son **un solo paso**. **Transición:** «Cómo se mantiene esa cuenta correcta lo explica Hugo.»

Parte 2 — Hugo (≈ 8 min)

13. Conteo ponderado (50 s) — Cada referencia tiene peso; el contador es la suma. Clonar reparte el peso **sin tocar el contador**. Tabla: X = 10 ⟶ clonar (6 + 4, hoja no se toca) ⟶ tirar Y (hoja 6) ⟶ … ⟶ hoja 0, se libera. No tocar la hoja elimina la carrera.

14. El peso en el atomic shared pointer (50 s) — Fig. 2: (a) cartel 104, dos pases, hoja 106; (b) tras un `load`, cartel 103 y un pase nuevo, hoja sigue en 106. Al crear: peso C ≫ P y se suma C a la hoja. Invariante: hoja ≥ peso + pases.

15. Empaquetar en 64 bits (45 s) — x86-64 usa 48 bits de dirección: 16 bits altos libres (19 con alineación). Restar un ticket = restar 2^{48}; la dirección no cambia y el acarreo sale por arriba. Contador del objeto de 64 bits (con 32 y C = 2^{19} bastarían 8192 carteles para desbordarlo).

16. Algoritmo 1 (55 s) — `store`: sumar C al nuevo *antes* de publicar; `swap` de golpe; restar al viejo **solo el sobrante** w. `load`: un `fetch_and_add` de −1 ticket; si quedan menos de P, recargar; devolver un pase ya pagado. El viejo se libera cuando le toca; no se retrasa.

17. Por qué desaparece la carrera (55 s) — Un `fetch_and_add` (F) y un `swap` (S2) tocan la misma palabra: solo hay dos órdenes. Caso 1: F antes ⟶ Ana tiene pase de A y la hoja deja 1 para ella. Caso 2: F después ⟶ Ana se lleva un pase de B y no toca A. No hay tercer caso.

18. Recargar el talonario (65 s) — Si el peso baja de P, ese hilo recarga: reserva C − w y sube el peso con un **CAS** (el cartel pudo cambiar de libro o alguien recargar ya). R-1: $C \geq 2P$. R-2: el peso solo baja de 1 en 1 o se fija por encima de P. Fig. 4: si el CAS falla y el peso subió, otro ya recargó ⟶ **salida temprana**.

19. Algoritmo 2 (45 s) — Reservar C − w **antes** del CAS; éxito ⟶ terminar; otro puntero o peso mayor ⟶ devolver la reserva y salir; peso menor ⟶ reintentar. Cadena de pesos estrictamente decreciente entre P−1 y 0: como mucho P intentos.

20. Ejemplo completo, P = 4, C = 8 (60 s) — Fila a fila: cuatro lecturas normales (8 ⟶ 4); Ana y Beto ofenden y recargan; Ana gana el CAS (2 ⟶ 7); Beto falla, ve 7 > 2 y **sale** devolviendo su reserva. Cierre: 7 + 6 = 13 = hoja.

21. Demo 2: el simulador (55 s) — 20 000 ejecuciones por variante, invariantes tras cada paso. Señalar **Paper: 0 fallos**; ingenua 4 796, umbral 0: 5 089, sin reserva 1 171, sin salida temprana 5 467, todas en 2 a 7 pasos. Contraejemplo de «sin reserva»: Carla hace `load`, Beto hace `swap` y resta toda la reserva ⟶ liberación con un lector dentro. Validación con 4 hilos: CAS 1: 80,1 %, 2: 18,2 %, 3: 1,7 %, 4: 0,06 % (cota P = 4). **Transición:** «Esto es evidencia; la demostración y la evaluación las cuenta Diego.»

Parte 3 — Diego (≈ 8 min)

22. La demostración (55 s) — El peso es de 16 bits sin signo: −1 sería 65 535 tickets fantasma. Ofensa = decrementar con peso ≤ P; O = multiconjunto de ofensores. Invariante 1: cada hilo está en O como mucho una vez. Invariante 2: peso ≥ P − |O|. Luego peso ≥ P − P = 0; por eso el umbral es P y no 0.

23. Coste y extensiones (50 s) — Bucle CAS ≤ P intentos. `load` suele ser una instrucción; **O(P) es el peor caso**. Con $C = 2^{19}$ y $P = 2^{16}$ hay ≈ 458 752 lecturas entre recargas. `null`, punteros débiles, *aliasing*; compare-and-swap y `load` *aliased* solo *lock-free*.

24. Experimentos del paper (50 s) — Pila concurrente, 144 hilos (rendimiento) y 56 (latencia). Con 1 hilo la más rápida; ≈ 7× Williams y ≈ 20× Folly con 1 pila y 144 hilos; pierde frente a Anderson con una sola pila (CAS *lock-free*).

25. Figura 6: latencia (35 s) — `load`, p = 56, mediana: Ours 1,14 µs; Anderson 2,58; Folly 11,6; Libstdc++ 99,7. Menor latencia y cola más corta.

26. Pruebas en local (1): el boceto (45 s) — P = 8, C = 16. Versión del paper: 0 errores y máx. 7 intentos (≤ P). Ingenua: 601 681 errores y −102 892 «sin liberar» (dobles liberaciones). Sin salida temprana: 0 errores pero 38 intentos > P; con C = P: 50 060 errores.

27. Pruebas en local (2): rendimiento (45 s) — i5-10400, 12 hilos, WSL. n = 1, u = 1: Autores 34,3 · Libstdc++ 2,2 · Mutex 6,4 Mop/s. **Matiz honesto**: con u = 50 el mutex gana (2,9 frente a 2,2) porque el CAS reintenta. Con 1 hilo, Autores es la más rápida.

28. Pruebas en local (3): latencia (40 s) — `load` (mediana / p99): Autores 269 ns / 0,83 µs; Libstdc++ 3,06 / 16 µs; Mutex 61 ns / **27,6 µs**. La mediana engaña; la cola delata a quien espera.

29. ¿Coincide con el paper? (40 s) — Se reproduce: 0 errores con ASan, la más rápida con 1 hilo, gana a GCC, mejor cola que el mutex. Matices: el mutex gana con 1 pila y 50 % de actualizaciones; 12 hilos y no 144; sin Folly/Williams/Anderson; un equipo y una ejecución.

30. Relación con el split reference count (35 s) — Contador local que sube desde 0; el paper cuenta lo contrario: peso = C – contador local. Lo nuevo es la recarga que **sale si otro ya recargó**: eso la hace *wait-free*. Folly pasa a bloqueante con más de C hilos.

31. Valoración crítica (40 s) — Fortalezas: simple, *wait-free*, sin memoria extra, liberación inmediata, lecturas de una instrucción. Límites: CAS y *aliasing lock-free*; cada lectura **escribe**; límite de P hilos (de corrección); depende de bits libres del puntero.

32. Conclusiones (45 s) — (1) *Wait-free* sin retrasar la liberación; (2) peso prepagado en el puntero; (3) recarga ≤ P intentos por R-1 y R-2; (4) comprobado en local. Dejar las dos preguntas de discusión.

Preguntas probables (respuesta corta)

Para David

1. *¿Por qué no un candado?* Es bloqueante: un hilo parado con el candado frena a todos (mutex: p99 de 27,6 µs).
2. *¿Por qué no un DCAS?* Hay que modificar dos posiciones separadas a la vez y el hardware actual no lo tiene.
3. *¿Con qué frecuencia ocurre la carrera?* Rarísima (nanosegundos), pero con ASan y 8 hilos la ingenua cae enseguida.
4. *¿Lock-free frente a wait-free?* Lock-free: siempre avanza *algún* hilo. Wait-free: **cada** hilo acaba en pasos acotados.
5. *¿Por qué no punteros de riesgo?* Gastan memoria extra y retrasan la liberación.
6. *¿Afecta a Rust o Swift?* Sí: el problema es el de cualquier conteo de referencias con variables compartidas mutables.

Para Hugo

1. *¿Qué es el peso?* El talonario del cartel: tickets ya apuntados en la hoja.
2. *¿Por qué $C \geq 2P$?* Tras recargar el peso queda ≥ P + 1 aun con P ofensores; con C = P el boceto dio 50 060 errores.
3. *¿Para qué la salida temprana?* Si el peso subió, otro ya recargó; sin ella hubo 38 intentos con P = 8.
4. *¿Y si el cartel cambia de libro?* El CAS compara también el puntero: falla, se devuelve la reserva y se sale.
5. *¿Caben puntero y peso en 64 bits?* Sí: 48 bits de dirección y 16 libres en x86-64.
6. *¿Por qué se reserva antes del CAS?* Para que la hoja cubra los tickets antes de publicarlos; sin reserva el simulador halla 1 171 liberaciones prematuras.

Para Diego

1. *¿Cuánto cuesta un `load`?* Normalmente una instrucción; O(P) solo en el peor caso.
2. *¿Por qué el umbral es P?* Es el colchón: peso $\geq P - |O| \geq 0$.
3. *¿Y con más de P hilos?* Se pierde la corrección; P es el máximo de hilos por variable.
4. *¿Por qué no 144 hilos?* Nuestro equipo tiene 12; coinciden las tendencias, no las cifras absolutas.
5. *¿Por qué a veces gana el mutex?* Con 1 pila y 50 % de actualizaciones el CAS *lock-free* reintenta; el paper lo reconoce.
6. *¿Por qué mediana y p99?* La mediana engaña (mutex 61 ns); la cola delata a quien espera (27,6 µs).

Errores típicos que evitar

- «El algoritmo retrasa la liberación»: **falso**; libera al llegar a 0.
- «Es O(P) siempre»: O(P) es el **peor caso**.
- «Todo es wait-free»: solo `load`, `store` y `swap`; CAS y *aliasing* son *lock-free*.
- Presentar los 144 hilos como medidas nuestras: las nuestras son de 12 hilos.
- «El mutex es peor en todo»: gana con u = 50 y en la mediana de `load`; pierde en la cola.
- Dar cifras sin contexto (hilos, n, u), sobre todo los 34,3 Mop/s.
- Confundir ofensa con error: ofender es bajar el peso estando en la zona roja; solo obliga a recargar.